CS3160 — Simulator Code Map

Each file in `src/`, its contents, and the lab in which it is written. Files belonging to later labs are absent from the folder; the folder as given always builds and runs. Every place to be completed is marked `TODO`, so `grep -rn TODO src/` lists the work of the lab.

The files

File	What's in it	Lab	Key types
`common.h`	`u32`, `i32` and friends; `sign_extend()`, `bits()`, `fatal()`	given	—
`log.{h,cpp}`	`log_out()`: one `[OUT]` line per retired instruction	given	`LogLevel`
`config.{h,cpp}`	reads and checks the `--config` file	given	`Config`, `CacheConfig`
`stats.{h,cpp}`	the counters, and `stats.json`	given†	`Stats`, `LevelStats`
`memory.h`	the four calls every memory level answers	given	`MemoryLevel`
`ram.{h,cpp}`	byte-addressed memory, and `--dump`	**lab 3**	`Ram`
`loader.{h,cpp}`	the `.r5ob` image into RAM	**lab 3**	—
`decode.{h,cpp}`	32 bits into a `Decoded`; `is_branch()`, `is_load()`, …	**lab 4**	`Decoded`, `Op`, `Format`
`disasm.{h,cpp}`	a `Decoded` into the text `objdump` prints	**lab 4**	—
`alu.{h,cpp}`	what the arithmetic does, and when a branch is taken	**lab 5**	`AluInputs`
`processor.{h,cpp}`	the registers, and the stage functions every model shares	**lab 5**	`Processor`, `Fetched`, `Executed`, `MemAccess`, `Latch`
`processor_single.cpp`	`run_single()`: one instruction per cycle	**lab 5**	—
`processor_pipelined.cpp`	`run_pipelined()`: five stages, four latches, the flush	**lab 6**	`Latch`
`hazard.{h,cpp}`	dependences between instructions, and their resolution	**labs 6–8**	`Hazard`, `Forwarding`, `FwdSrc`
`processor_forwarding.cpp`	`run_forwarding()`: operands forwarded between stages	**lab 8**	—
`cache.{h,cpp}`	one cache level: lookup, replacement, the write path	**labs 9–10**	`Cache`, `CacheBlock`, `CacheAddr`
`sim.cpp`	`main()`: the options, the machine it builds, the run	given	—

† `stats.cpp` is given except `level_amat()` (Lab 10). The three `processor_*.cpp` are deliberately separate run loops: three machines executing the same instructions, whose `stats.json` can be compared directly.

What you write, lab by lab

Lab	File	Functions
3	`ram.cpp`, `loader.cpp`	`ram_read` `ram_write` `ram_dump` `loader_load`
4	`decode.cpp`, `disasm.cpp`	`decode` `disasm`
5	`alu.cpp`, `processor.cpp`, `processor_single.cpp`	`alu_apply` `branch_taken` `stage_execute` `next_pc_of` `stage_memory` `stage_writeback` `run_single`
6	`processor_pipelined.cpp`, `hazard.cpp`	the five stages, the flush and the commit inside `run_pipelined`; `redirects_pc` `wrong_path_instructions`
7	`hazard.cpp`	`hazard_stalling`
8	`processor_forwarding.cpp`, `hazard.cpp`	the EX and ID halves of `run_forwarding`; `hazard_forwarding` `forward_in_ex` `forwarded_value`
9	`cache.cpp`	the geometry switch in `cache_init`; `cache_split` `cache_block_addr` `cache_find` `cache_order_insert` `cache_order_touch` `cache_choose_way` `cache_evict_for` `cache_fill` `Cache::read`
10	`cache.cpp`, `stats.cpp`	`cache_writeback_if_dirty` `Cache::write` `Cache::read_block` `Cache::write_block` `cache_flush` `level_amat`

The structs, in the order one instruction meets them

```
Fetched     raw, pc, latency              what came back from imem
Decoded     op, fmt, rs1, rs2, rd, imm    what those 32 bits mean
Executed    result, taken                 the ALU's answer; result is a value, an
                                          address, or a branch target
MemAccess   did_read, did_write, addr,    what the load or store did
            value, latency
Latch       valid, d, v_rs1, v_rs2, e,    all of the above, for one instruction in
            next_pc, m                    flight -- lab 6 onwards
```

`Processor` holds the state that persists between instructions: `pc`, `regs[32]`, and the `imem` and `dmem` it accesses. `Stats` holds the measured counters. Both are given in Lab 5 and remain unchanged thereafter.

Where the memory hierarchy plugs in

`MemoryLevel` is the interface: `read`, `write`, `read_block`, `write_block`. `Ram` implements it, as does `Cache` from Lab 9, with a `next` pointer to the level below. The processor calls `imem->read()` without knowing whether a cache is present, so adding one changes no code written in Labs 3–8.

```
cpu.imem -> I1 -> RAM             an instruction cache
cpu.dmem -> L1 -> L2 -> RAM       a two-level data side
```